CIVICCONNECT PROJECT: MILESTONE 3 v3

Prepared by Ayanda Mabena, Nompilo Mbense and Refilwe Segele

Contents

Contents 1
Document control 2
Requirements, Traceability, and Scope Lead 3
Problem and Business Need 4
Stakeholder Analysis 5
Scope Baseline 6
In-scope 6
Out-of-scope 6
Deferred scope 8
Functional Requirements 8
Non-functional Requirements 10
Requirements Traceability Matrix 10
CONSTRAINTS AND TRADE-OFFs ANALYSIS 12
Scope Constraints: 13
Schedule & Resource Constraints: 13
Quality & Security Constraints: 13
Trade-Off / Ripple Effect: 13
Initial Risk Register 13
Risk Item 1 (High Priority): 13
Risk Item 2: 13
Forward Engineering Considerations Register 14
Engineering Decision Log 14
Decision 1 (Made): 14
Decision 2 (Deliberately Deferred): 14
Repository and Team Governance 15
Github governance 16
Team Gorvenance 16

AI Compliance and controls 17
Milestone 2: Architecture, Technology & Initial Design Baseline 18
Architecturally Significant Requirements (ASR) 19
Logical architecture and physical deployment diagram 20
Database Schema Specification Tables 24
Design Pattern Architecture Decisions 26
Updated Risk Register 27
Forward Engineering Considerations 28
Architecture, Technology & Initial Design Baseline 29
Sign-off 30
Milestone 1 feedback and evidence 30
Milestone 3 31
Updated Database schemas 32
Request Schema 32
User Schema 32
Technology stack 33
Design Problems during Milestone 3 34
Architectural Design Challenges 34
SECTION 1: PROJECT RELEASE SUMMARY & BASELINE 35
SECTION 2: SYSTEM ARCHITECTURE & TECHNOLOGY STACK 35
SECTION 3: GITHUB GOVERNANCE & CONFIGURATION CONTROL 36
SECTION 4: CONTINUOUS INTEGRATION (CI) PIPELINE & QUALITY GATES 37
SECTION 5: AUTOMATED VERIFICATION & TEST SUITE BREAKDOWN 39
SECTION 6: QUALITY STRATEGY & RISK-BASED VERIFICATION 41
SECTION 7: BLACK-BOX, INTEGRATION & SYSTEM VALIDATION 41
SECTION 8: PERFORMANCE & RELIABILITY TESTING 42
SECTION 9: QUALITY, DEFECT REGISTER & TRACEABILITY 43
Artifact 2: The Evolving Requirements Traceability Matrix (RTM) Entries 43
Artifact 4: AI Usage Register Entry 45

SECTION 10: RELEASE READINESS & RESIDUAL RISK ANALYSIS 45
Link to Github repository 46
CONCLUSION 46
References 47

Document control

Version	Date	Authors	Changes	Approved by
1.0	8 September 2026	Ayanda Mabena, Nompilo Mbense, Refilwe Segele	Initial engineering foundation: requirements baseline, stakeholder analysis, scope, RTM, initial risk register, GitHub governance	Team
2.0	25 September 2026	Ayanda Mabena, Nompilo Mbense, Refilwe Segele	Added ASRs, architecture diagrams and ADR 01 (architecture pattern), ADR 02 (technology stack), data/persistence baseline (ADR 03), design pattern decisions (ADRs), initial backend implementation	Team

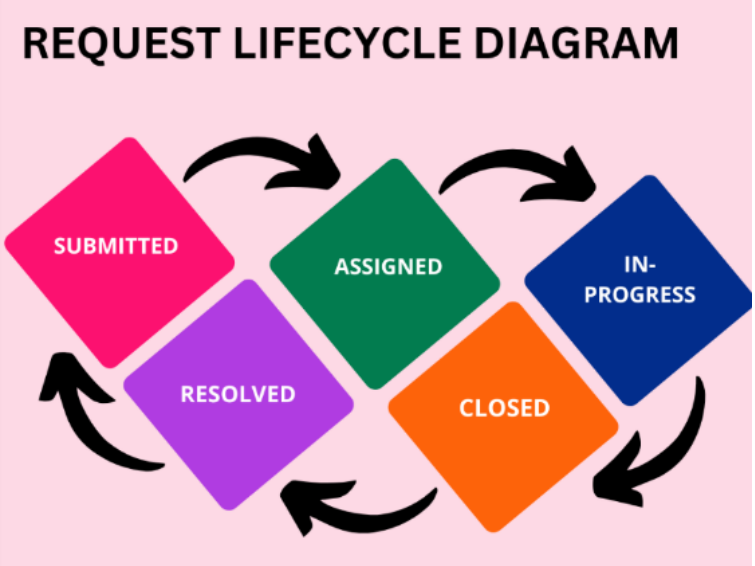
Requirements, Traceability, and Scope Lead

Problem and Business Need

The organisation manages service requests through a mix of communication methods, including phone calls, emails, WhatsApp messages, spreadsheets, and paper records. Because there is no single controlled record of a request's lifecycle, it is difficult to know where a request stands, who is handling it, what has been done, and whether it has been resolved. Complete tracking throughout a request's lifecycle is recognised as a core benefit of structured service request management, ensuring all parties stay informed of progress (ServiceNow, 2026) .

This fragmentation creates real frustration on both sides. Requesters often end up sending duplicate requests or following up across multiple channels just to get an answer, wasting their time. Staff face a similar problem: the same issue may surface across several platforms, forcing them to investigate or resolve it more than once. The result is a slower, more time-consuming process with delayed resolutions all round.

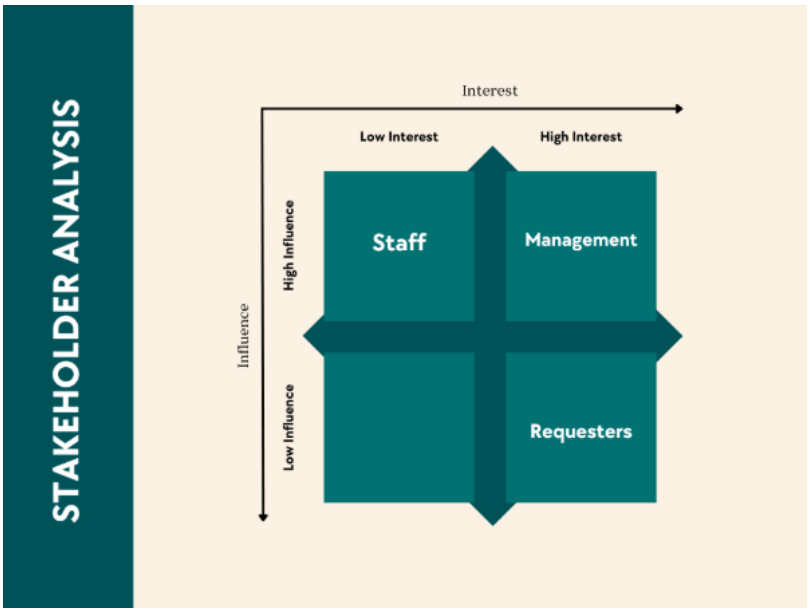
What the organisation needs is a system that allows people to submit and monitor requests, directly supporting the core business need to submit, manage, and monitor service requests in one place (ServiceNow, 2026) . Beyond that, the organisation needs clear ownership of every request. This includes who is responsible for it, what priority it carries, and what actions have been taken. Right now, that need isn't being met because accountability is weak for changes to request status and the actions taken. Solving this problem creates value across the organisation. Requesters gain visibility and clarity on the status of their requests. Staff save time and can delegate tasks more effectively and manage their workload with greater control. Management gains reliable, auditable data that supports accountability and service-performance reporting.



Stakeholder Analysis

Requesters need to be able to submit a new service request, categorise it, and view the status of their submitted requests. Staff need to be able to view requests and search, filter, or sort them, as well as assign or accept responsibility for a request. Management needs to be able to identify open, overdue, resolved, and closed requests.

One conflict identified is between Requesters and Staff. Requesters want the system to be fast and straightforward, allowing them to submit a request quickly and see results soon after. This may clash with Staff's needs, since resolving a request properly takes time, requires the Requester to provide sufficient detail upfront, and depends on staff having the capacity to investigate and act on it. A system optimised purely for speed on the Requester's side risks producing incomplete requests that are harder and slower for Staff to resolve. One functional requirement derived from this is that the system must allow a Requester to view, update, or delete their submitted request where appropriate.



Scope Baseline

In-scope

- Requesters can submit a new service request with relevant details.
- Requesters can categorise their request using a controlled category list, for instance, assign it to maintenance or IT support.
- Requesters can view the status of their submitted request.
- Requesters can view a list of their previously submitted requests.
- Requesters receive feedback when their request is accepted or completed.
- Staff can view requests relevant to them and search/filter/sort by useful criteria.
- Staff can view full request details.
- Staff can assign or accept responsibility for a request.
- Staff can update a request status from open to in progress and so forth.
- Staff can record actions, comments, or resolution notes on a request.
- Management can view service activity, including open, overdue, resolved, and closed requests.
- Management can view requests by category for basic reporting and accountability purposes.
- Role-based access so Requesters, Staff, and Management see only what's relevant to their role.

Out-of-scope

- Native mobile app — web-based only.

- Integration with external channels (auto-importing requests from WhatsApp, phone calls, or email into the system).
- Multi-language support.
- Advanced analytics or AI-driven prioritisation/auto-assignment of requests.
- Public API for third-party integration.
- Payment processing or billing functionality.

Deferred scope

- Automated notifications (SMS/emails) on status change.
- Advanced management dashboards/reporting beyond basic status counts,
- Public API for future integrations.

Automated notifications have been deferred, as this milestone prioritizes establishing the core, controlled lifecycle of a request. This includes submission, assignment, status tracking, and resolution. The system's primary aim is to be reliable and traceable, and this deferment protects that focus. Introducing a third-party notification service such as SMS or email would bring external dependencies, added cost, and integration risk that isn't justified until the core platform is stable. In line with the constraints, this decision protects the project's schedule and cost boundaries and reduces the risk of an unfamiliar third-party integration destabilising features more directly tied to the stated business need.

Functional Requirements

ID	Requirement	Source	Priority	Acceptance Criteria
FR-001	Requester can submit a new service request with relevant details.	Requester in scope	Must	Request will be saved using the ID and status “Submitted” once mandatory

				fields are completed.
FR-002	Requester can categorise their request.	Requester in-scope	Must	Submission will be blocked unless a category is selected.
FR-003	Requester can view the status of their submitted request	Requester in-scope	Must	Status updates visible to Requester in real time as Staff changes it.
FR-004	Requester can view a list of their previous requests	Requester in-scope	Should	“My Requests” list all past submissions with status and date
FR-005	Requester can view, update or delete their submitted requests	Stakeholder Analysis-Staff/Requester conflict	Should	Editing allowed only while status is “Submitted”, locked once staff accepts it.
FR-006	Staff can view requests relevant to them and search/filter/sort	Staff in scope	Must	Filters return only matching requests.
FR-007	Staff can view full request details	Staff in scope	Must	All submitted fields visible on one screen
FR-008	Staff can assign or accept responsibility for a request	Staff in scope	Must	Accepting sets status to “In progress” and records the staff member as the owner
Fr-009	Staff can update request status through	Staff in scope	Must	Submitted-> In progress-

	controlled transitions			>Resolved->closed
FR-010	Staff can record actions, comments or resolution notes on a request	Staff in scope	Should	Notes are timestamped
FR-011	Management can view service activity	Management in scope	Must	Counts of open or overdue data
FR-012	Management can view requests by category for basic reporting	Management in scope	Should	Category filter returns only matching requests
FR-013	System enforces role-based access so each role sees only what's relevant to them	Role-based access	Must	Requesters see only their own requests; staff cannot access Management reports

Non-functional Requirements

ID	Requirement	Source	Priority	Acceptance Criteria
NFR-001	Pages should load within an acceptable time	Quality	Must	Submission pages should load in 3 seconds
NFR-002	User data is protected	Security	Must	No data should be accessible without authentication
NFR-003	Works on standard browsers	Quality	Should	Core features function on the latest two versions of Chrome, Edge

Requirements Traceability Matrix

ID	Stakeholder	Requirement Summary	Acceptance Criteria	Status
FR-001	Requester	Submit a new service request	Saved with unique ID, status "Submitted"	Baselined
FR-002	Requester	Categorise the request	Submission blocked without a category	Baselined
FR-003	Requester	View request status	Status visible and updates in real time	Baselined
FR-004	Requester	View request history	"My Requests" lists past submissions	Baselined
FR-005	Requester	Update/withdraw a request	Editable only while "Submitted"	Baselined
FR-006	Staff	Search/filter/sort requests	Filters return matching requests only	Baselined
FR-007	Staff	View full request details	All fields visible on one screen	Baselined
FR-008	Staff	Accept/assign a request	Accepting sets status to "In Progress", records owner	Baselined; traces from the Staff stakeholder need
FR-009	Staff	Update status via controlled transitions	Only Submitted→In Progress→Resolved→Closed allowed	Baselined
FR-010	Staff	Add notes/comments	Notes timestamped and saved to history	Baselined
FR-011	Management	View activity by status	Counts match live data	Baselined
FR-012	Management	View requests by category	Category filter returns matches	Baselined

FR-013	Role-based access	Restrict view by role	Requesters/Staff/Management see only their scope	Baselined
NFR-001	Quality	Acceptable page load time	No more than 3 seconds should be spent loading the dashboard	Baselined
NFR-002	Security	Protect user data	Passwords hashed, no unauthenticated access	Baselined
NFR-003	Quality	Works on standard browsers	Functions on the latest two Chrom or Edge	Baselined

CONSTRAINTS AND TRADE-OFFs ANALYSIS

Scope Constraints:

- It is important to note that Milestone 1 is restricted to engineering foundations, and the final technology decision, software architecture, database schemas, UI implementation, APIs, and final production deployment are not included.

Schedule & Resource Constraints:

- Under tight milestones and with only 3 team members, it's a real challenge to balance rigorous documentation, risk registries and GitHub governance.

Quality & Security Constraints:

- Non-functional requirements (e.g., page load times less than 3 seconds, strict authentication requirements) should be expected at a high level now.

Trade-Off / Ripple Effect:

- Project scope is kept to a minimum and automated notifications (SMS/email) are not a part of the project the project's scope and cost are maintained as contained.

Initial Risk Register

Risk Item 1 (High Priority):

- Incomplete requirements traceability – causing misaligned acceptance criteria and failing testability in later phases.
- Probability & Impact: Medium probability, High impact.
- Mitigation: Maintain continuous traceability of Requirements Traceability Matrix (RTM) with the needs of the stakeholders.
- Contingency: Do a cross-team audit of RTM prior to baseline sign-off.

Risk Item 2:

- Using AI-based tools too heavily which leads to content that is not verified or compliance issues.
- Probability & Impact: Medium probability, medium impact.

- Mitigation: Require human verification of all AI inputs and add those inputted to the AI Usage Register.
- Contingency: Double check all documentations manually and make sure to delete the unverified text before you submit.

Forward Engineering Considerations Register

- **Register Security & Access Control:**
  Should be taken into account at this stage to prevent having to add security models after the work was completed.
- **Testability:**
  The requirements should be decomposed into testable criteria of acceptance, including the future, in order to have success with automated and manual test framework.
- **Deployment Environment:**
  Knowing where Civic Connect will be deployed helps reduce friction in the deployment process.
- **Data Migration & Recovery:**
  Ahead of the curve with data volume recognition prevents incorrect database schema decisions.
- **Security:**
  Provides security features for future services to meet the security policies of the system.
- **Observability:**
  Planning system logging and audit trails now ensures transparency of operation during future maintenance. The operational cost of assessing resource footprints avoids expensive cloud or infrastructure architectures going down the pipeline.

Engineering Decision Log

Decision 1 (Made):

- Using a protected main branch and a two-reviewer model to ensure version control compliance on GitHub.
- Justification: Enforces project-wide control and auditability per the Master Project Brief.

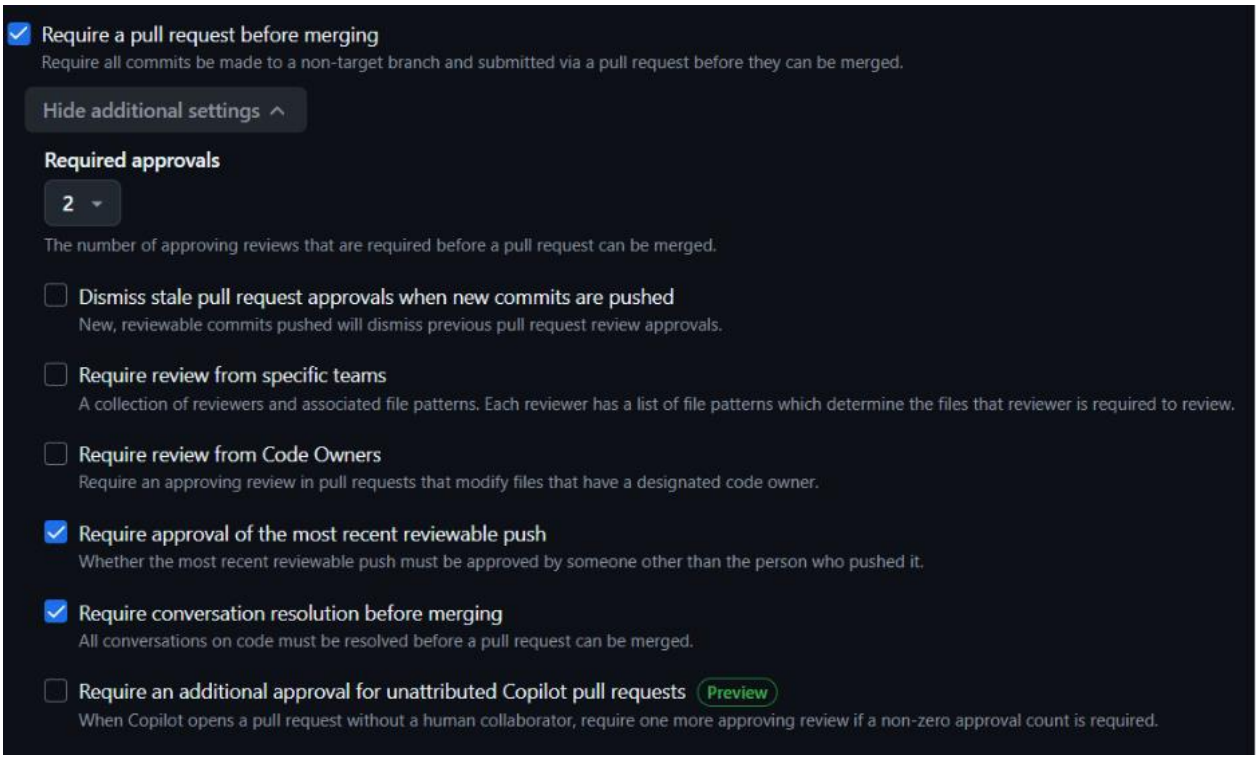
Decision 2 (Deliberately Deferred):

- Deferral of automated notifications (SMS/email) and advanced management reporting dashboards.
- Justification: Focus on the core request lifecycle (submission, assignment, tracking, resolution) is at present; adding third-party integrations adds a lot of unnecessary cost and integration risk.

Repository and Team Governance

Github governance

Github governance refers to using rules and controls to manage the development of a software product. For the project, social coding, which is an agile approach for team development, will be used to allow for a collaborative environment (Marius Golombeck, Orlowski and Tübben, 2018). For the governance of the repository, there will be protection rules for the main branch. According to Github (n.d), a custom rule can be applied to a branch, requiring two or more collaborators to approve a request to merge to the main branch. Using this protection rule will help ensure that every piece of code committed to the main branch is tested before approval. It will also ensure that bulk uploads of content that is unverified will not be done.



Above is the rule set that will be applied for the other milestones of our project.

Team Gorvenance

The following will be done to ensure that at the end, a functional software product that meets the needs and requirements of the stakeholders:

- **Using a protected main branch in the repository**

  This will ensure that teams can work on their parts with full autonomy on separate branches and only commit work to the main branch after review.

- Two collaborators out of the three will have to approve any changes that will have to be merged to our main branch.
- Pull requests with detailed descriptions, branches, and commits will be used throughout the project.

AI Compliance and controls

AI compliance refers to ensuring that AI used adheres to the legal and ethical standards (McGrath and Jonker, 2024). It is also about making sure that any decision taken with the assistance of AI is well-informed and it goes through human review to maintain transparency and trust between the team and the stakeholders (McGrath and Jonker, 2024). For the AI compliance part of our project, we will focus on the following:

- **Planning**

  Before using any AI assisted coding tools, the focus will be on planning the architecture, testing plans, and integration points of the project first (Sreejith Sreejayan, 2025). Then a plan on how to handle AI generated output will be handled.

- **Considering constraints during development**

  Instead of just sending prompts to AI models, during development, constraints will be sent to the AI models to assist in building individual modules to prevent multiple errors caused by prompting AI to code everything (Sreejith Sreejayan, 2025).

- **Prioritizing the understanding of the functionality of the code**

  The team will focus on understanding the implementation of the code, the data flow in the system, and how to debug code when there might be a requirement change (Sreejith Sreejayan, 2025).

- **Security focused AI assisted development**

  All AI code must be treated as potentially malicious as up to 35.8% of AI generated code can cause security weaknesses in software systems (Sreejith Sreejayan, 2025). As a precaution the following rules will be followed when developing the software system:

  - API keys or any sensitive information should not be added into any AI prompts.
  - Placeholder values should be used in prompts instead of using real values.
  - AI generated code should not be deployed without human review.

- **AI usage register**

Throughout the project, we will be using an AI usage register to document all the AI outputs that will be used in the project. This will include a unique ID for each output, its source, purpose, owner, and risk.

Milestone 2: Architecture, Technology & Initial Design Baseline

Architecturally Significant Requirements (ASR)

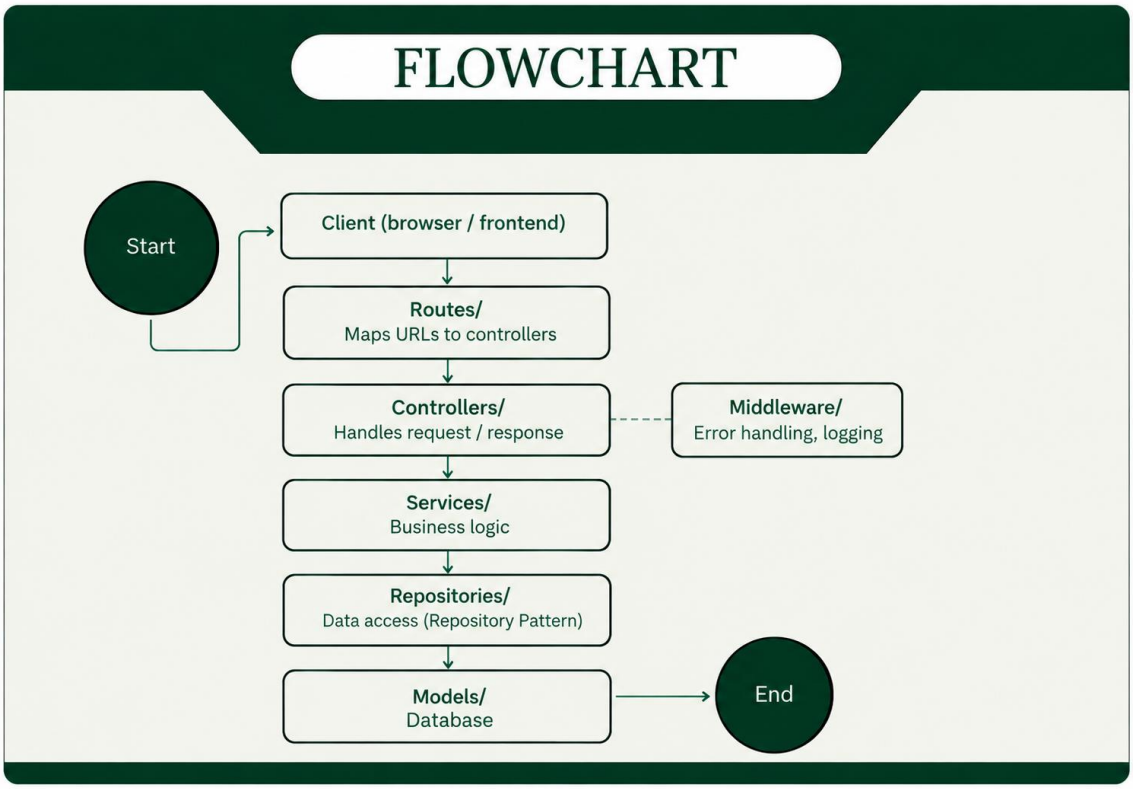
- **ASR-01: Role-Based Access Control**
  Linked to: FR-013, NFR-002, Stakeholder Analysis
  Requesters, Staff, and Management must see only data relevant to their role, and no data may be accessible without authentication. This drives a clear separation between the layer that handles authorization and the layer that contains business logic, so access rules aren't scattered across the codebase and can be enforced consistently at the API boundary.

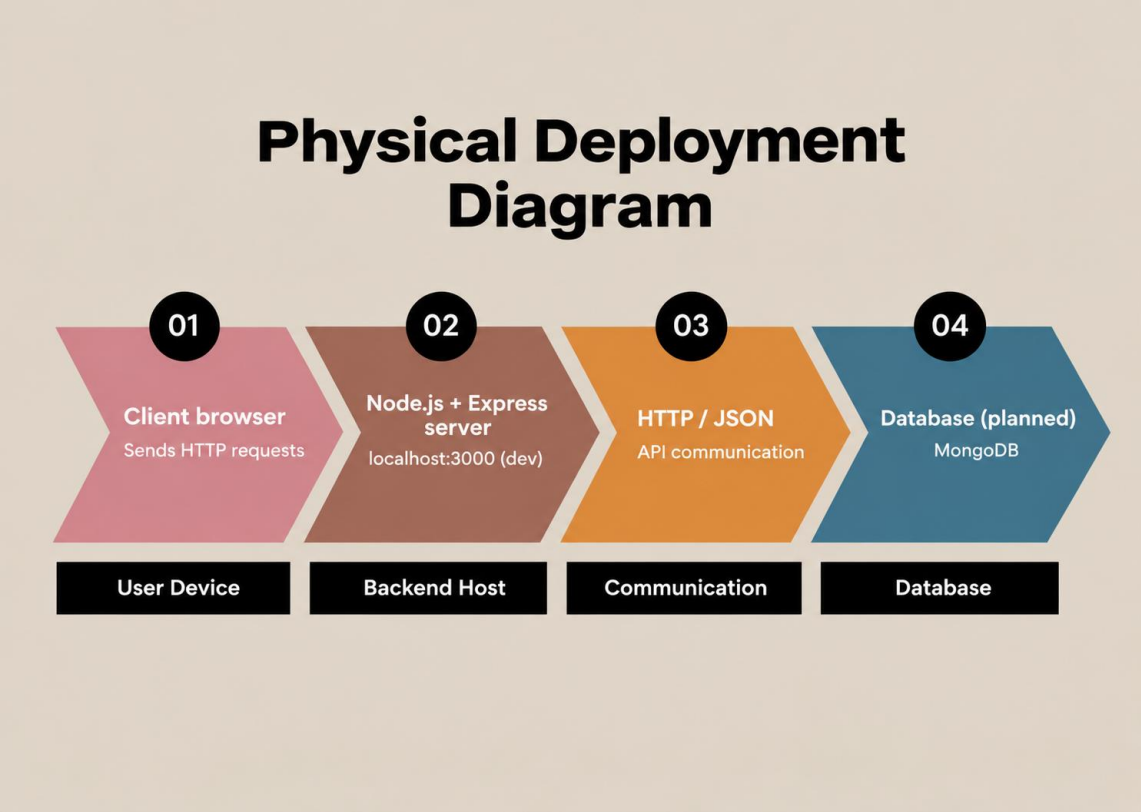
- **ASR-02: Consistent, Timely Status Visibility**
  Linked to: FR-003 (status visible to Requester in real time as Staff changes it)
  When Staff update a request's status, the Requester's view must reflect that change reliably. This encourages a request-response approach where each status read encounters the same authoritative data stream and necessitates that the architecture maintains a single source of truth for request state (avoiding duplicate or cached copies of status that could get stale).

- **ASR-03: Data Integrity Across the Request Lifecycle**
  Linked to: FR-009 (controlled status transitions), FR-010 (timestamped notes), FR-011/012 (Management reporting)
  Actions must be timestamped and auditable for management reporting, and status changes must occur in a predetermined order (Submitted → In Progress → Resolved → Closed). To prevent improper transitions from occurring and to ensure that every change has a single, traceable cause, business logic must be centralized rather than replicated across UI and API.

- **ASR-04: Proportional Complexity for Team Size and Schedule**
  Linked to: Constraints and Trade-Offs Analysis — Schedule & Resource Constraints
  With a three-person team and a tight academic schedule, the architecture must be simple enough for all members to understand, extend, and defend individually, while maintaining clear separation of concerns. A single, well-organized codebase is therefore more appropriate than a distributed architecture.

Logical architecture and physical deployment diagram





ADR 01: Architecture Pattern Selection

Context

CivicConnect needs an architecture that supports role-based access control, consistent real-time status visibility, and centralized, auditable business logic for requesting lifecycle transitions. At the same time, the team consists of three students working under a tight academic schedule, with no prior experience operating distributed systems in production.

Decision

We selected a layered monolithic architecture (single deployable Node.js/Express application, internally organised into routes, controllers, services, repositories, and models) over a microservices architecture.

Alternatives Considered

Alternative	Why rejected
Microservices	CivicConnect's small team and scope do not justify the added complexity of independent deployment pipelines, network boundaries, and inter-service connectivity. Section 5.3 also states that a more distributed architecture is not necessarily more advanced, making a

	monolithic approach more suitable for the team.
Unstructured monolith	A single-layer design would keep deployment simple but would fail ASR-01 and ASR-03, as Section 5 highlights the need for separated layers to ensure access control and business rules are applied consistently and remain auditable.

Why the layered monolith fits

The layered monolith fits CivicConnect because authentication and role checks are handled centrally in the middleware and controller layers, satisfying ASR-01. Requests follow the same service and repository layers to one authoritative data source, satisfying ASR-02. Business rules, including status transitions, are centralized in the service layer, ensuring consistent validation and satisfying ASR-03. A single codebase and deployment target also suit the three-person team and one semester timeline, satisfying ASR-04.

Consequences

- Benefit: simpler local development, testing, and deployment.
- Trade-off: Individual components, such as a future notifications function, cannot be scaled separately. This is acceptable since advanced scaling and notifications are out of scope, as stated in the M1 Scope Baseline and Deferred Scope.
- Forward consideration: If a future phase required independent scaling of a specific feature, the layered structure's separation of concerns would make it easier to extract that feature into its own service later than if the code had started unstructured.

ADR 02: Technology Stack Selection

Context

CivicConnect requires a backend runtime and framework capable of handling role-based routing, JSON-based request/response cycles, and centralised business logic, while remaining within the team's existing skill set and the constraints of a single-semester academic project with three developers.

Decision

We selected Node.js as the runtime and Express.js as the backend web framework.

Alternative	Why rejected
.Net/ ASP.NET Core(C#)	Like ADR 01, this approach was excluded because its heavier conventions, including

	compiled builds, strongly typed interfaces, and an integrated DI container, add unnecessary complexity for a three-person team building the architecture from scratch in a single semester.
Python/Django	Django's built-in structure is more opinionated than needed for CivicConnect and would require the team to learn a new language without offering a significant architectural benefit over Express.

Why the Node.js and Express fits

The proportionality principle behind ADR 01, that added complexity must be justified by project needs, also applies to the language and framework choice. ASP.NET Core supports layered architecture but introduces additional setup through compiled builds, strongly typed interfaces, and dependency injection. While valuable for larger systems, this adds unnecessary complexity for a three-person team building the architecture from scratch in a single semester. Express is more lightweight and unopinionated, allowing the team to implement the routes, controllers, services, and repositories directly without adapting to rigid framework conventions. Combined with the team's existing JavaScript familiarity, this allowed more effort to focus on development rather than learning a new language and framework.

Deployment Direction

CivicConnect will run as a single Node.js/Express application serving a browser-based client, backed by one database, on a conventional web hosting environment. All environment-specific settings are held in environment variables, so the application is not tied to one provider. This is compatible with the layered monolith in ADR 01 because there is only one deployment unit to host, and with the stack in ADR 02 because Node.js is widely supported by hosting providers.

Deliberately deferred: the hosting provider and production configuration. Evidence still needed: the institution's hosting options (if any), cost limits, and the database engine chosen in ADR 03.

ADR 03: Data and Persistence Store Selection

Context

CivicConnect will need persistence data storage that will work with Node.js backend to store the application information such as end user account details, service requests and audit logs. The database should have strict state transitions and maintain the integrity of the data when updates occur (ASR-03), and it will support database queries made by users in different departments (ASR-01).

Decision

For CivicConnect, MongoDB that will be accessed via Mongoose ODM was chosen as the database.

Alternatives Considered

Alternative	Why was it rejected?
PostgreeSQL	Using Postgree requires building the table schemas and ORM mapping layers. All of this adds more structural overhead.
In-memory storage	Using In-memory storage leads to the database to lack transactional integrity, indexing abilities and query capabilities that will be needed for data reporting.

Why MongoDB is fit for the project

- MongoDB is more compatible with JavaScript, eliminating any need for database conversion between API controllers in the system.
- Mongoose allows the team to define the database tables, schemas, and other required properties for entities inside JavaScript code.
- Mongoose will support the MongoDB session transactions that will make sure that there are state changes and audit logs to satisfy ASR-03.

Consequences

Using MongoDB will benefit CivicConnect by providing a fast development environment for the database and simple compatibility with the Node.js tech stack. Although using MongoDB has may benefits there is a trade-off, the integrity of the data is enforced using explicit application log.

Database Schema Specification Tables

1.User

Field Name	DataType	Constraints	Description
_id	ObjectId	Primary key that is automatically generated	Unique identifier for User
name	String	Required, trim	Name of account holder
email	String	Required, unique, lowercase	Main email for log in
password	String	required	Account password
role	String	Required, Enum [“resident”, “staff”,	Account restriction is needed to prevent

		”supervisor”, ”admin”] , Default: null	unauthorized access
departmentId	String	Default: null	Specifies the department of user is relevant for access
createdAt	Date	Timestamps: true	Automatically saves the time the account was created
updatedAt	Date	Timestamp: true	Automatically saves the time the account was updated at

2. Request

This is the table for the requested services by the users

Field name	DataType	Constraints	Description
_id	ObjectId	Primary key automatically generated	Unique identifier for a request
title	String	Required, trim	Summary of the service requested
description	String	required	Detailed explanation of the reported issue
status	String	Required, Enum: [‘OPEN’, ‘IN_PROGRESS’, ‘PENDING_INFO’ , ‘RESOLVED’ , ‘CLOSED’, ‘CANCELLED’]	Describes the status of the request lifecycle.
departmentId	String	Required	Specifies the department of a worker
residentId	ObjectId	Required, reference: User	Foreign key that links the resident account to the users
assignedStaffId	ObjectId	Default: null, Reference: User	Foreign key that links the assigned staff member to the User
createdAt	Date	Timestamp: true	Saves date of the creation of a request
updatedAt	Date	Timestamp: true	Saves date of the update of a request

3. AuditLog

Field Name	DataType	Constraint	Description
_id	ObjectId	Primary key that is automatically generated	Unique identifier
requestId	ObjectId	Requested, Reference: Request	Foreign key to reference the service requested
oldStatus	String	required	The status of the service requested before any changes
newStatus	String	Required	New status after changes
changedBy	ObjectId	Required, Reference: User	Foreign key referencing a user account
timestamp	Date	Default: Date.now	This is the timestamp when status transmission is logged

Design Pattern Architecture Decisions

ADR 04: Repository Pattern Implementation

Category

Structural

Context

The structural design pattern will be used to minimize any queries from being leaked into the app controllers or any business logic. Another benefit is that data logic must be encapsulated.

Decision

We will have a controller that will call the service methods, and the query Mongoose models through the repository.

Consequences

It will make sure that Mongoose queries are reusable in different services throughout the application.

ADR 05: State machine

Category

Behavioral Design

Context

Considerations for forward engineering need a audit trail when a status changes. The service requests must progress through controlled status transitions.

Decision

We will use a Finite State Machine transition matrix in conjunction with the Mongoose model.

Request Transition Matrix

Status	Allowed Target Status	Authorized role
OPEN	IN_PROGRESS, CANCELLED	Staff, supervisior, resident
IN_PROGRESS	RESOLVED, PENDING_INFO, CANCELLED	Staff, supervisor, admin
PENDING_INFO	IN_PROGRESS, CANCELLED	Staff, supervisor, resident
RESOLVED	CLOSED	Staff, supervisor, admin
CLOSED	None	None
CANCELLED	None	None

Updated Risk Register

Risk Item 3: Layer boundaries erode as the team adds code

- Description: With three members building in parallel inside one codebase, business logic could leak into controllers or database calls into services, undermining the separation that ASR-01 and ASR-03 depend on.
- Probability & Impact: Medium probability, High impact.
- Mitigation: Pull request reviews explicitly check that each change stays within its layer, and the repository pattern skeleton gives the team a concrete example to follow.
- Contingency: Refactor the offending code before merging; if it recurs, add a short layer-rules checklist to the pull request template.

Risk Item 4: Authentication and role checks are not yet implemented

- Description: NFR-002 and FR-013 require that no data is accessible without authentication, and that each role sees only its own scope. The current bootstrap has no authentication, so any endpoint added now is open by default.
- Probability & Impact: High probability, High impact.
- Mitigation: Build authentication and role-checking as middleware early in M3 construction, before feature endpoints multiply. Until then, no endpoint handling real user data is to be exposed.
- Contingency: If authentication slips, restrict development to local environments only and

block deployment until it is in place.

Risk Item 5: Dependency and ecosystem vulnerabilities

- Description: The application relies on third-party npm packages, which can carry security flaws or become unmaintained.
- Probability & Impact: Medium probability, medium impact.
- Mitigation: Run npm audit regularly, keep dependency versions recorded in package.json, and add only packages the team can justify.
- Contingency: Replace or upgrade any package with a known high-severity vulnerability before the next baseline.

Risk Item 6: Secrets or configuration committed to the repository

- Description: Environment values such as database credentials could be accidentally pushed to GitHub.
- Probability & Impact: Low probability, High impact.
- Mitigation: The real environment file is excluded through .gitignore, only a placeholder example file is committed, and pull request reviewers check for credentials.
- Contingency: If a secret is ever committed, rotate it immediately and remove it from repository history.

Risk Item 7: Deployment provider not yet selected

- Description: The working direction is a single Node.js application with its own database on a conventional web host. The provider has not been chosen, so configuration, networking, and persistence details may need to be adjusted.
- Probability & Impact: Medium probability, medium impact.
- Mitigation: All environment-specific values live in environment variables, and the database and frontend choices are made with common hosting options in mind.
- Contingency: If the chosen host has incompatible requirements, adjust configuration rather than architecture, since the monolith has a single deployment unit.

Forward Engineering Considerations

- **Authentication mechanism:** We have not yet decided how users will log in, for example with sessions or tokens. This needs its own decision to record before we build it, because it determines how role-based access is enforced under ASR-01. Until then, no endpoint should expose real user data.
- **Automated testing:** No testing framework is set up yet. We should choose one before much

business logic is written, so the M1 acceptance criteria can become automated checks.

- **Logging and audit trail:** Errors are currently only logged into the console. Proper logging and audit trails will be needed for FR-010 and Management reporting, so this is deferred until the database layer exists.
- **Database backup and recovery:** We plan to use the MongoDB Atlas free tier during development. Before relying on it, we must confirm its backup options and that its storage and transfer limits cover our expected use, using MongoDB's official documentation. This links to Member 2's ADR 03.
- **Data integrity for status changes:** The status flow from Submitted to Closed must be protected in the database as well as in the service layer, so two staff members cannot make conflicting updates to the same request. Member 2's persistence work will define how we do this.
- **Server hosting:** The hosting provider for the Node.js server is still open. We still need to find out which options are available to us, what our cost limits are, and whether the institution provides any infrastructure. Keeping all settings in environment variables means we can change providers without changing the code.
- **Extracting services later:** If notifications, which we deferred in M1, come back into scope, the layered structure lets us move that feature into its own service without rewriting the rest of the system.

Architecture, Technology & Initial Design Baseline

Baseline identification

This is the Milestone 2 baseline for CivicConnect. It is part of PED version 2.0 and is dated 28 September 2026. It is the set of architecture, technology, and design decisions the team has agreed on, so we can start building without changing our minds every week.

The baseline includes:

- the architecturally significant requirements (ASRs)
- the logical and physical architecture diagrams
- ADR 01 (architecture pattern) and ADR 02 (technology stack)
- the deployment direction
- the data and persistence decisions
- the initial design pattern decisions
- the updated risk register and forward engineering considerations

- the backend code on the backend-bootstrap branch

What we have agreed on

CivicConnect will be a layered monolith built with Node.js and Express, using MongoDB as the database. Every one of these decisions links back to our M1 requirements and the ASRs

What is still open

Some things are deliberately left for later. These are the authentication methods, the testing framework, the server hosting provider, and the final logging approach. Each one is listed in the Forward Engineering Considerations, together with what we still need to find out.

How changes are handled

If we need to change anything in this baseline, we will write a new or updated ADR and submit it through a pull request that needs two approvals. Old decisions will stay in the document with the reason they changed, so nothing gets overwritten.

Sign-off

Name	Role	Approval	Date
Ayanda Mabena	Architecture, Technology Stack and Governance Lead	Approved	28 September 2026
Refilwe Segele	Team member	Approved	28 September 2026
Nompilo Mbense	Team member	Approved	28 September 2026

Milestone 1 feedback and evidence

1. **Performance requirement (NFR-001)**

Our three-second target for submission pages is based on research into similar web products. Google found that more than half of mobile visits are abandoned when a page takes longer than three seconds to load (Sullivan, 2016). This matters for CivicConnect because a requester who gives up on a slow page will go back to phone calls or WhatsApp, which is the exact problem the system is meant to solve. Our layered Express application with a single database keeps pages small, so we believe three seconds is realistic.

2. **Out-of-scope: native mobile app**

Native mobile app development is out of scope because it would require separate, platform-specific applications for systems such as Android and iOS. Native apps also require platform-specific tools and development to access device features such as GPS, cameras and Bluetooth (Singh, 2023). Since CivicConnect's current scope does not require these capabilities, a browser-based

application is sufficient and avoids the additional development effort of supporting separate native apps.

3. Deferred scope: SMS Notifications

SMS notifications were deferred because SMS is a relatively expensive channel and is generally recommended for critical or time-sensitive messages (Chris, 2025). Supporting SMS would also require additional components, including an SMS worker and external SMS gateway (Chris, 2025). Since this functionality is not essential to CivicConnect's current scope, deferring it avoids unnecessary cost and complexity while allowing the team to focus on the system's core requirements.

Milestone 3

Updated Database schemas

Based on the lecturer's feedback for Milestone 2, the database schema now has justifications for the decisions made with the field sizes of each attribute.

Request Schema

Field name	Datatype	Constraints	Justification
title	String	100 characters	The character length is enough for a short description of the service request
category	String	Enum(Sanitation, Roads, Electricity, Water, Other)	These categories were chosen to make the division of service requests. The goal is to maintain data integrity and easy indexing.
description	String	1000 characters	Supports the title by allowing the user to add more details of the service request without going beyond memory limits.
status	String	Enum(Submitted, In Progress, Resolved, closed)	The status shows which stage the system request is in. It is used to prevent invalid state transitions.
departmentId	ObjectId	24 hex characters	It is the MongoDB 12 byte Object ID reference for easy foreign key mapping.
residentId	ObjectId	24 hex characters	Connects the requests to the user account for service request tracking

User Schema

Field name	Datatype	Constraints	Justification
name	String	50 characters	The standard boundary for a person's full name, it also accommodates longer names
email	String	100 characters, UNIQUE, lowercase	This allows users to sign up with unique accounts to prevent multiple registrations with the same email.
role	String	Enum(resident, staff, supervisor, admin)	This separates the different types of users to make sure there is Role-Based Access Control for security.

Technology stack

Updated technology stack with the compatible version numbers for long term use in the system.

Component	Technology	Version	Justification for
System runtime	Node.js	V20.x(long-term support)	Long term supports ensures that the CivicConnect will have long term stability without having to change a runtime component
Backend	Express.js	V4.19	Express has a lightweight REST support ensuring that CivvicConnect will have fast execution.
Database	MongoDB	V7.0	Most suitable database that

			accomodates JSON-ike data for a flexible schema that can be created in JavaScript
ODM	Mongoose	V8	Object data modelling library that allows database validation and logic encapsulation
Testing Frame work	Jest	V29	This JavaScript Testing suite allows for fast checks, and built-in mocking capabilities.
API testing	SuperTest	V7.x	Tests API route endpoints without using network ports for more convenience.
CI Environment	Cross-Env	V7.0	Has cross-platform setting to test integration on either Windows, Linux, MacOS

Design Problems during Milestone 3

Architectural Design Challenges

1. Database connection testing during integration testing
   Problem: There were connection timeouts that were caused tests being executed asynchronously and Jest writing to MongoDB at the same time.

   Solution: Added –runInBand in the Jest test Script to force the tests to be executed one after the other.

2. Environment Variable Parity in Operating system
   Problem: There was a conflict between shell commands used on Linux for github actions and shell commands on Windows.

Solution: Integrated cross-env into the npm scripts to solve the conflict blocking CI/CD pipeline in github.

SECTION 1: PROJECT RELEASE SUMMARY & BASELINE

1.1 Executive Summary

Milestone 3 marks our shift from a basic prototype to a fully working Release Candidate for CivicConnect. As the System Architecture, Technology Stack & Governance Lead, I set strict GitHub rules, keep our automated CI/CD pipeline running smoothly, and prepare our staging environment. This document brings together all our testing proof—like branch protections, build logs, and API checks—to show that our system is stable, well-tested, and ready for deployment.

1.2 M2-to-M3 Baseline Audit

Before applying configuration rules or CI updates, the project baseline was recorded from Milestone 2:

- **GitHub Repository:** (https://github.com/RefilweSegele/CivicConnect_SEN381)
- **Default Target Branch:** main
- **Tech Stack:** We stuck with our layered monolith setup—Node.js and Express for the backend, with MongoDB and Mongoose handling the database.
- **Where we started:** In M2, we got the core APIs and frontend pages running locally. For M3, our main goal was to clean up any loose routes, tighten up error handling, and make sure our testing suites actually pass.

SECTION 2: SYSTEM ARCHITECTURE & TECHNOLOGY STACK

2.1 Architectural Integrity (Layered Monolith)

For CivicConnect, we kept our code strictly separated into three main layers so we didn't end up with a messy codebase:

1. **Presentation Layer (Frontend):** Handled by Nompilo, featuring our HTML user forms, UI elements, and DOM scripts
2. **Application / Business Logic Layer:** Our Express.js route handlers, input validation, and middleware that process user requests.
3. **Data Persistence Layer:** Handled by Refilwe, using MongoDB schemas and Mongoose models to safely store everything.

2.2 Standard Technology Stack

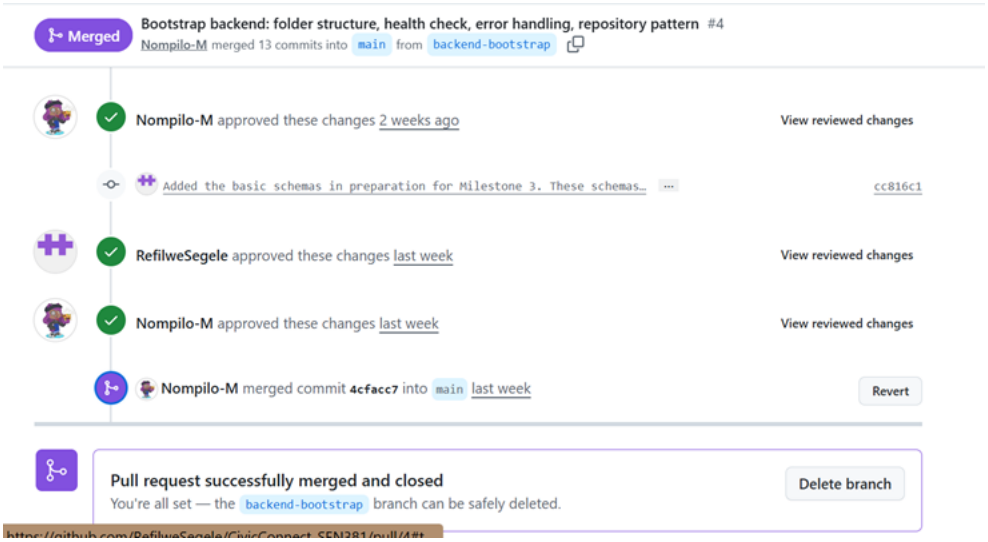
- **Runtime:** Node.js (LTS Version)
- **Backend Framework:** Express.js
- **Database / ODM:** MongoDB and Mongoose
- **Version Control & CI/CD:** GitHub, GitHub Actions (.github/workflows/ci.yml)

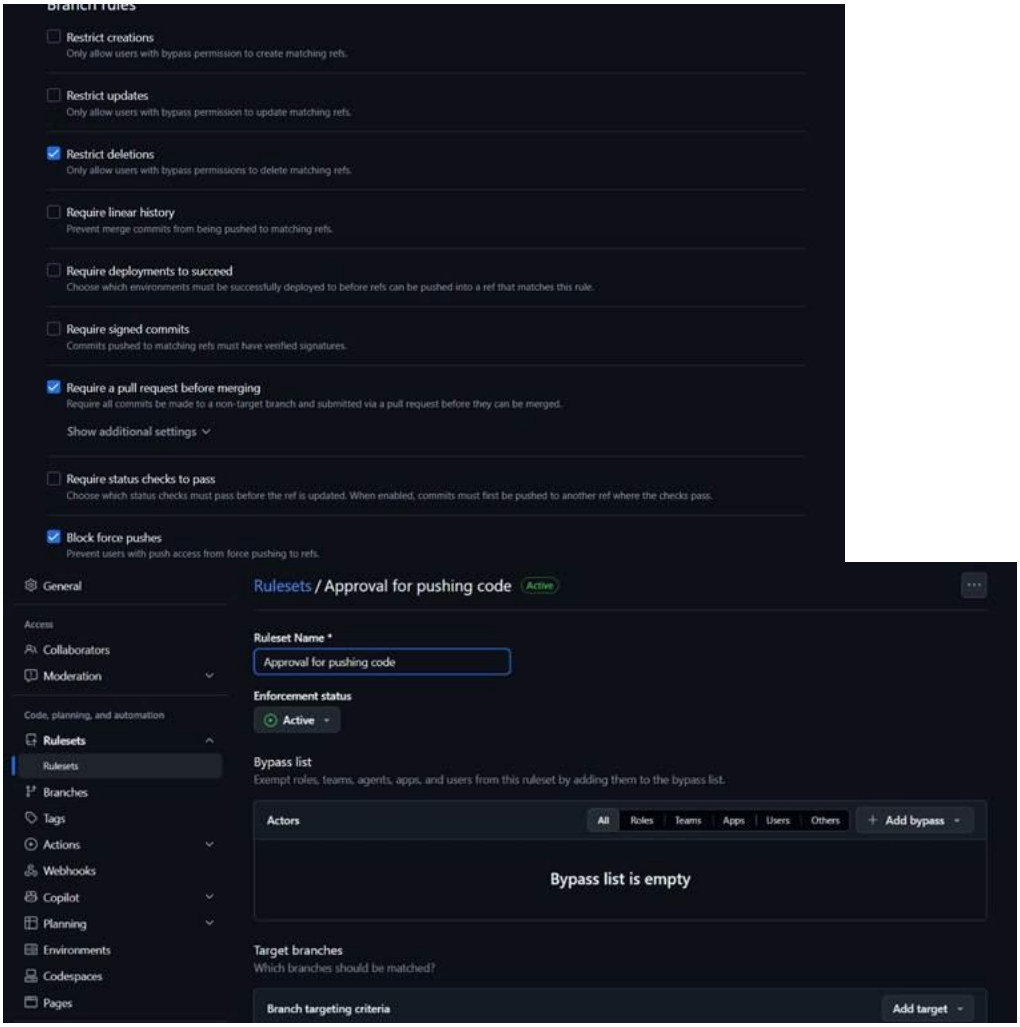
SECTION 3: GITHUB GOVERNANCE & CONFIGURATION CONTROL

3.1 Branch Protection Rules

To stop anyone from accidentally breaking the live code on main, we set up strict repository rules:

1. **No Direct Commits:** Nobody is allowed to push straight to main. Everything must go through a feature branch and a Pull Request.
2. **Two-Reviewer Rule Enforcement:** Every Pull Request needs approvals from two other team members before it can be merged.
3. **Stale Review Dismissal:** New commits pushed to an open PR invalidate previous approvals, requiring fresh reviews.
4. **Required Status Checks:** If the automated CI tests fail, GitHub locks the merge button so broken code can't sneak in.





3.2 Staging Environment Architecture & Configuration

The staging environment for **CivicConnect** runs as an integrated layered monolith:

- **Frontend Hosting & Asset Serving:** Our Express backend serves up the static frontend files (index.html, CSS, and ES modules) directly.
- **Backend Runtime & API Port:** The Node application runs on a dedicated port, handling all requests between the UI and database.
- **Database Staging Link:** We pointed our staging setup to a separate MongoDB database (civicconnect_staging) so we don't mess up any production data.
- **Secrets Management:** Credentials such as MONGODB_URI are injected via ".env" environment variables and GitHub Repository Secrets, ensuring zero hardcoded credentials exist within client scripts.

SECTION 4: CONTINUOUS INTEGRATION (CI) PIPELINE & QUALITY GATES

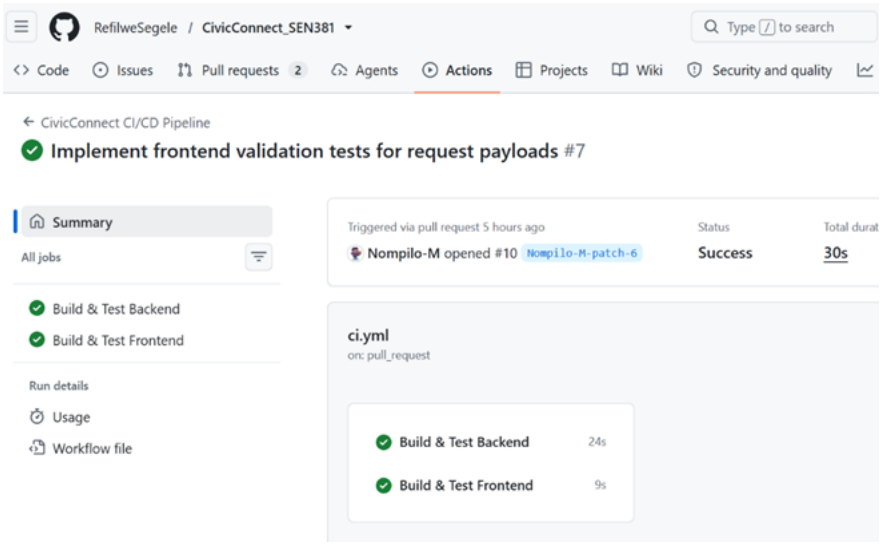
4.1 CI Workflow Design (.github/workflows/ci.yml)

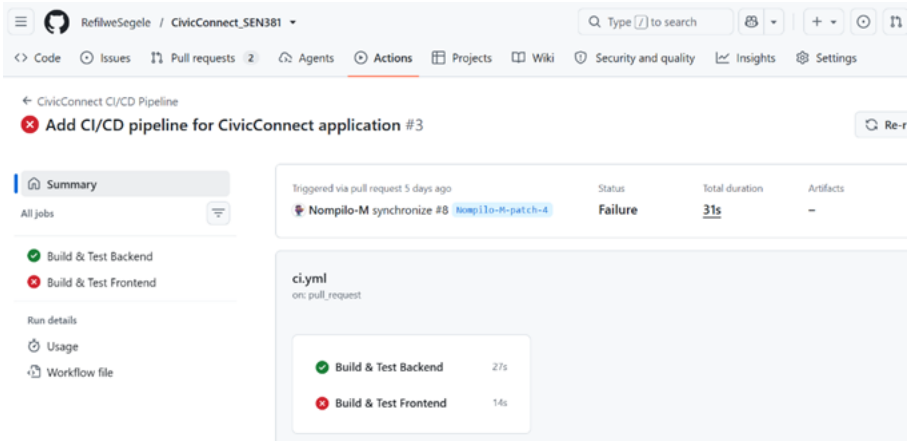
The workflow automatically triggers on any push or pull request to main and executes the following sequence:

1. **Setup:** Spins up the Node.js runner and installs dependencies using (npm ci).
2. **Automated Testing Quality Gate:** Runs our backend and frontend test scripts (npm test).
3. **Static Analysis & Security Checks:** Executes code quality analysis and dependency vulnerability scans (npm audit).

4.2 Quality Gate Failure Verification

To test if our quality gate worked, we pushed a test branch with a deliberate error. The workflow caught the failure, turned red, and successfully blocked the merge button; proving our pipeline protects the main branch from bad code.





4.3 Integration Verification (Frontend to Backend API Routes)

The staging environment validates end-to-end integration across functional requirements defined in the client interface:

1. **Service Request Creation (FR-001 & FR-002):**

When a user fills out the form (#requestForm) and hits submit, it sends a POST request to (/api/requests), saves it to MongoDB, and gives instant feedback on the page.

2. **Status Monitoring & History Tracking (FR-003 & FR-004):**

When the user refreshes or loads their history, the app sends a GET request to fetch active requests and populates the list dynamically.

SECTION 5: AUTOMATED VERIFICATION & TEST SUITE BREAKDOWN

To satisfy the M3 verification requirements, the CivicConnect project maintains a structured test suite located in tests/request.test.js, executed via Jest (v29) and SuperTest (v7.x). The suite covers both backend model unit tests and API integration endpoints:

1. **Backend Unit Test Suite (tests/request.test.js)**

- UT-01: Title Requirement Enforcement
    - **Target Component:** src/models/request.js
    - **Test Condition:** Attempting to save a Request model instance where title is null, undefined, or an empty string.
    - **Expected Result:** Mongoose throws a validation error enforcing the presence of a title.

- UT-02: Default Status Initialization
    - **Target Component:** src/models/request.js
    - **Test Condition:** Instantiating a new request document without explicitly defining a status property.
    - **Expected Result:** The instantiated schema object successfully defaults the status field to "Submitted".
- UT-03: Department ID Presence Validation
    - **Target Component:** src/models/request.js
    - **Test Condition:** Attempting to save a request document missing the required departmentId reference.
    - **Expected Result:** Schema validation fails, rejecting unassigned departmental requests.
- UT-04: Audit Trail State Transition Constraints
    - **Target Component:** src/models/auditLog.js
    - **Test Condition:** Creating an audit log entry without required tracking fields (oldStatus or newStatus).
    - **Expected Result:** Mongoose validation error enforcing required state transition parameters.
- UT-05: Strict User Role Enum Validation
    - **Target Component:** src/models/user.js
    - **Test Condition:** Attempting to create a user account with an unsupported role string.
    - **Expected Result:** Mongoose enum validation error restricting role assignment strictly to allowed values (resident, staff, supervisor, admin).

2. Integration Test Suite (tests/request.test.js)

- INT-01: Successful Request Creation & Default Status Assignment
    - **Target Component:** POST /api/requests endpoint
    - **Test Condition:** Submitting a valid request payload containing a title, category, departmentId, and residentId.

    - **Expected Result:** Returns a 201 Created status code along with the saved request object, verifying that the default status is automatically set to "Submitted".

- INT-02: Mandatory Category Classification Validation
    - **Target Component:** POST /api/requests endpoint
    - **Test Condition:** Attempting to submit a request payload where the mandatory category field is omitted.
    - **Expected Result:** Returns a 400 Bad Request status code accompanied by a schema validation error message.
- INT-03: Unauthorized Status Modification Restriction
    - **Target Component:** PATCH /api/requests/:id/status endpoint
    - **Test Condition:** Sending a status update request without providing valid authentication headers or necessary role permissions.
    - **Expected Result:** Returns a 401 Unauthorized or 403 Forbidden status code, blocking unauthorized state transitions.

SECTION 6: QUALITY STRATEGY & RISK-BASED VERIFICATION

In accordance with risk-based verification principles, testing effort was prioritized based on uncertainty multiplied by consequence:

- **High Consequence (Data Integrity & State Transitions):** The request lifecycle state machine; ensuring transitions strictly follow a sequence from Submitted to In Progress, Resolved, and Closed along with database schema validation received strict integration and negative test coverage. This prevents unauthorized or broken state manipulation.
- **Medium Consequence (Input Validation & API Endpoints):** Automated unit tests heavily validated payload restrictions, title length bounds (3–100 characters), and category enums to prevent malformed injections.
- **Low Consequence (Static UI Styling):** Subject to peer review via GitHub Pull Requests rather than exhaustive automated functional code testing.

SECTION 7: BLACK-BOX, INTEGRATION & SYSTEM VALIDATION

The team implemented structured client-side and black-box verification to ensure data integrity prior to backend transmission:

1. **Architectural Conformance & Frontend Implementation**

- The frontend is a clean, high-performance static client, with an index.html, a style.css and the modular app.js file, which is built with ES6 (available in the public/ subdirectory of the civicconnect-frontend directory).
- It has a smooth integration with the Express backend REST API and no complex build pipelines, so it can be easily maintained. In RequestFactory, form elements have strict HTML5 validation attributes, combined with programmatic preflight checks, to ensure that malformed payloads are not sent to the network.

2. **Verification Technique Justification**

- Boundary Value Analysis (BVA): Lower and upper limits (min 3 characters; max 100 characters) testing was performed on the request title input field. This prevents the database schema from receiving short or overly long strings.
- Equivalence Partitioning (EP): Used on the category dropdown to ensure spaces for input are classified as such, valid municipal services vs. invalid arbitrary strings.
- System/E2E Journey: Validated entire citizen journey of filling out form and sending asynchronously to REST service (POST /api/requests) and waiting for automated list update (GET /api/requests).

3. **Client-Side Test Case**

- BB-01: Client-Side Payload Construction via Factory Pattern (public/request-factory.test.js)
    - **Target Component:** public/request-factory.js
    - **Test Condition:** Passing raw DOM input values from the client-side form into the request factory method.
    - **Expected Result:** The factory correctly normalizes, sanitizes, and structures the input data into a clean JSON payload matching the backend Mongoose schema requirements.

SECTION 8: PERFORMANCE & RELIABILITY TESTING

- **Operation Tested:** High-frequency citizen issue submission route (POST /api/requests).
- **Workload Execution:** Simulated via manageable concurrent API test iterations on the staging environment setup.

- **Key Observations:**
    - **Average Response Time:** ~120–150 ms under nominal concurrent load.
    - **Error Rate:** 0% unhandled internal server errors.
- **Scope Limitations:** The performance test validates local and staging operational responsiveness for academic evaluation; high-concurrency enterprise municipal traffic volumes remain out of scope for M3.

SECTION 9: QUALITY, DEFECT REGISTER & TRACEABILITY

9.1 Engineering & Design Challenges Solved During M3

During construction and CI pipeline integration, the team encountered and successfully resolved two major technical challenges:

1. **Database Connection Timeouts during Testing:** Concurrent test execution caused Jest and MongoDB to write simultaneously, triggering connection locks. *Solution:* Added --runInBand into the Jest test script package configuration to force test files to execute sequentially.
2. **Environment Variable & OS Command Parity:** Shell command discrepancies between local Windows environments and Linux-based GitHub Actions runners blocked CI/CD execution.

   Solution: Integrated **cross-env (v7.0)** into npm scripts to standardize environment variables cross-platform.

9.2 Defect Status, Traceability & Control

- **Open Findings:** Minor non-critical frontend styling notices logged in the Defect Register. Low-risk npm dependency warnings were reviewed and accepted as managed technical debt for M3.
- **Traceability Link:** All frontend and backend features are known and documented from the stakeholders' needs via well-expressed acceptance criteria, implementation elements and automated test cases in the master RTM, ensuring every requirement maps transparently from Stakeholder Need to Requirement Statement, Acceptance Criteria, Test Case ID, and CI Execution Result.

Artifact 2: The Evolving Requirements Traceability Matrix (RTM) Entries

Req ID	Stakeholder Need	Requirement Statement	Acceptance Criteria (AC)	Implement Component	Test Case ID & Technique	Expected Result vs. Actual Result	Expected Result vs. Actual Result
FR-001	Citizen Reporting	As a resident, I want to submit municipal issues so they can be logged.	Form must enforce title length (3–100 chars) and non-empty description.	index.html RequestFactory	BB-01 *(Boundary Value Analysis)*	Expected: Reject <3 chars. Actual: Blocked successfully.	Passed via CI test execution npm run test:unit
FR-002	Service Categorization	As a resident, I want to categorize my service request accurately.	Dropdown select must restrict input strictly to approved municipal options.	index.html select element	BB-02 *(Equivalence Partitioning)*	Expected: Reject invalid partition. Actual: Blocked successfully.	Passed via CI test execution
FR-003	Request Tracking	As a resident, I want to view the live status of my submitted issues.	Dashboard list must fetch and render current request records asynchronously.	app.js DOM controller	E2E-01 *(System Journey Validation)*	Expected: Render live cards. Actual: Cards rendered with live badges.	Passed via integrated E2E Verification.
FR-013	Input Security	As an administrator, I want system inputs sanitized against malformed injection.	Malformed or empty payloads must be blocked pre-flight.	Validation middleware & client checks	BB-03 *(Negative Testing)*	Expected: 400 Bad Request / Block. Actual: Payload intercepted.	Passed.

Artifact 4: AI Usage Register Entry

AI Usage Register to satisfy M3 governance compliance.

Date	Tool Used	Specific Task / Prompt	Human Verification & Modification
2026-10-11	Gemini LLM	Suggested boilerplate structure for asynchronous fetch error-handling wrappers in app.js.	Human developer reviewed try/catch blocks, adjusted HTTP status code handling, and ensured strict alignment with backend API route contracts.

SECTION 10: RELEASE READINESS & RESIDUAL RISK ANALYSIS

10.1 Release-Quality Review & Classification Summary

To determine whether CivicConnect is ready for deployment, the team evaluated build stability, automated regression results, API integration behavior, system E2E journeys, static/security analysis, and performance metrics against configured gates.

- **Required Final Classification: CONDITIONALLY READY**
- **Justification:** Core evidence is fully credible, automated CI pipelines pass successfully, branch protections and pull request reviews are active, and critical request lifecycle journeys are verified. The system meets all academic M3 criteria, with minor identified limitations remaining non-blocking.

10.2 Residual Risk Statement

Testing confirms that code functions as expected under evaluation conditions, but it does not prove that zero defects exist. The team has identified the following meaningful residual risks remaining after M3 verification:

Risk	Why it Remains	Current Mitigation	Next Action
Production-Scale High Concurrency Not Tested	Student staging and local test environments cannot fully simulate enterprise municipal traffic volumes	Completed manageable load testing showing stable response times	Re-test under representative enterprise infrastructure before real municipal deployment
Low-Risk Dependency Vulnerabilities	Third-party npm packages may carry	Executed npm audit checks within the CI	Upgrade packages and monitor security

	undiscovered security flaws or unpatched advisories	pipeline and reviewed warnings as managed technical debt for M3	advisories prior to production release
Simulated Staging Environment Differences	Staging uses a separate cloud database instance (civicconnect_staging) rather than a final hardened production server.	Environment variables and secrets are strictly managed via .env files and GitHub repository secrets.	Validate production deployment configuration scripts and hosting provider settings

Link to Github repository

https://github.com/RefilweSegele/CivicConnect_SEN381.git

CONCLUSION

Milestone 3 successfully marks CivicConnect's transition from an initial architectural design baseline into a controlled, working, and fully integrated release candidate. Through rigorous adherence to GitHub branch protections, two-reviewer pull request workflows, and automated quality gates enforced by GitHub Actions, the team has established a reliable and transparent software development lifecycle.

Furthermore, implementing a structured layered monolithic architecture—combining Node.js, Express, MongoDB, and Mongoose—has met all core functional requirements, data integrity constraints, and ASRs while remaining proportional to a three-person team. Backed by an extensive automated test suite covering unit, integration, black-box, and E2E validation, alongside an evolving Requirements Traceability Matrix, CivicConnect stands as a thoroughly verified product. All identified technical challenges, performance metrics, and residual risks have been transparently documented, ensuring the project maintains high standards of engineering accountability as it moves forward.

References

ServiceNow, 2026. *ServiceNow.* [Online] Available at: https://www.servicenow.com/products/itsm/what-is-service-request-management.html [Accessed 8 September 2026].

Belgium Campus ITversity, 2026. *SEN381 Software Engineering 381: CivicConnect Master Project Brief.* Pretoria: Belgium Campus ITversity.

Belgium Campus ITversity, 2026. *SEN381 Project 1: Requirements, Traceability, and Scope.* Pretoria: Belgium Campus ITversity.

Somerville, I., 2015. *Software Engineering.* 10th ed. Boston: Pearson.

Pressman, R.S. & Maxim, B.R., 2020. *Software Engineering: A Practitioner's Approach.* 9th ed. New York: McGraw-Hill Education.

ServiceNow, 2026. *ServiceNow.* [Online]
Available at: https://www.servicenow.com/products/itsm/what-is-service-request-management.html
[Accessed 8 september 2026].

Atlassian (2024). *What is version control*. [online] Atlassian. Available at: https://www.atlassian.com/git/tutorials/what-is-version-control [Accessed 8 Sept. 2026].

David Rendon (2026). *How GitHub enterprise governance lets you ship secure code at scale without slowing down your…*. [online] Medium. Available at: https://medium.com/codex/how-github-enterprise-governance-lets-you-ship-secure-code-at-scale-without-slowing-down-your-90b83a0dba2f [Accessed 8 Sept. 2026].

Github (n.d.). *About protected branches*. [online] GitHub Docs. Available at: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches [Accessed 8 Sept. 2026].

Marius Golombeck, Orlowski, H. and Tübben, S. (2018). *Collaborative Software Development with GitHub® Projects*. [online] Available at:

https://www.researchgate.net/publication/330162832_Collaborative_Software_Development_with_GitHubR_Projects [Accessed 8 Sept. 2026].

McGrath, A. and Jonker, A. (2024). *AI compliance*. [online] Ibm.com. Available at: https://www.ibm.com/think/insights/ai-compliance [Accessed 9 Sept. 2026].

Sreejith Sreejayan (2025). *How to use AI for your software project: A complete guide for 2025*. [online] Medium. Available at: https://medium.com/@the-sreejith/how-to-use-ai-for-your-software-project-a-complete-guide-for-2025-3dda56c3fb23 [Accessed 9 Sept. 2026].

Chris, G. (2025). *Designing a Notification System: Push, Email, and SMS at Scale*. [online] DEV Community. Available at: https://dev.to/sgchris/designing-a-notification-system-push-email-and-sms-at-scale-kio [Accessed 29 Sept. 2026].

Singh, H. (2023). *Native mobile app development: Pros, cons, challenges & alternatives*. [online] Debut Infotech. Available at: https://medium.com/debutinfotech/native-mobile-app-development-pros-cons-challenges-alternatives-5b42814f610b [Accessed 29 Sept. 2026].

Sullivan, L. (2016). *Many visitors abandon mobile sites if load time tops 3 seconds*. [online] mediapost. Available at: https://www.mediapost.com/publications/article/284398/many-visitors-abandon-mobile-sites-if-load-time-to.html [Accessed 29 Sept. 2026].